

# A COMPREHENSIVE MEASURE OF WELL-BEING PREDICTION USING MACHINE LEARNING

## Project Description:

A Comprehensive Measure of Well-Being is an AI and Machine Learning-based application designed to predict the overall well-being score of a country using socio-economic, health, education, and environmental indicators.

The application enables users to enter values such as Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, Gross National Income (GNI) per capita, Happiness Score, Healthcare Index, Internet Usage, Employment Rate, and other relevant indicators. The trained Machine Learning model analyzes these inputs and predicts the country's Overall Well-Being Score.

The project is developed using Python, Flask, HTML, CSS, JavaScript, Scikit-learn, Pandas, and deployed using Render.

## Scenarios

**Scenario 1:** A government analyst wants to estimate the well-being score of a developing country after improving healthcare and education. The application predicts the updated well-being score instantly.

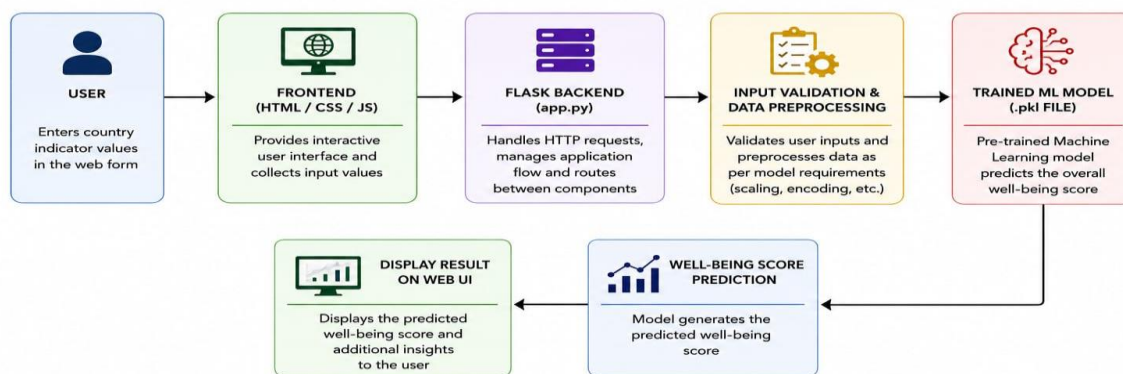
**Scenario 2:** A research organization compares different countries by changing economic indicators and observes how the overall well-being changes.

**Scenario 3:** A student studying data science enters sample values from a dataset to understand how different development indicators influence the well-being score

## Technical Architecture:

### TECHNICAL ARCHITECTURE

A Comprehensive Measure of Well-Being Prediction Using Machine Learning



**DESCRIPTION:** The user enters values of key socio-economic, health, education, and environmental indicators through the web interface. The Flask backend receives the data, validates and preprocesses it, and sends it to the trained Machine Learning model. The model predicts the comprehensive well-being score, which is then displayed to the user on the web interface.

**Description:** The frontend collects user inputs through an interactive web interface. The Flask backend validates the inputs, preprocesses them according to the trained model requirements, and sends them to the Machine Learning model. The model predicts the Comprehensive Measure of Well-Being score, which is displayed to the user in real time.

## Pre-requisites:

- Python Programming Knowledge
- Machine Learning Fundamentals
- Flask Framework
- HTML, CSS, and JavaScript
- Pandas and NumPy
- Scikit-learn
- Data Preprocessing Techniques
- Pickle (Model Serialization)
- Git & GitHub (Version Control)
- Visual Studio Code (VS Code)
- Render (Cloud Deployment)
- Web Browser (Google Chrome or Microsoft Edge)

## Project Workflow:

### Milestone 1: Data Collection and Model Development

- **Activity 1.1: Collect the Dataset**  
Collect a well-being dataset containing socio-economic, health, education, and environmental indicators.  
Verify the dataset quality and ensure all required features are available.
- **Activity 1.2: Data Preprocessing**  
Clean missing values and remove duplicate records.  
Perform feature engineering, normalization, and encoding where necessary.  
Split the dataset into training and testing sets.
- **Activity 1.3: Train the Machine Learning Model**  
Train suitable regression models such as Random Forest Regressor.  
Compare model performance using evaluation metrics.  
Select the best-performing model based on prediction accuracy.
- **Activity 1.4: Save the Trained Model**  
Save the trained model using Pickle (.pkl).  
Test the model locally before integrating it with the web application.

### Milestone 2: Core Functionalities Development

- **Activity 2.1: Develop Prediction Functionality**  
Load the trained Machine Learning model.

Accept user input for all well-being indicators.

Predict the Comprehensive Well-Being Score.

- **Activity 2.2: Implement the Flask Backend**

Create Flask routes for the home page and prediction endpoint.

Validate user inputs before prediction.

Return the predicted result to the frontend.

### **Milestone 3: App.py Development**

- **Activity 3.1: Develop the Main Application Logic**

Load the trained model during application startup.

Define Flask routes for home and prediction.

Process user input and generate prediction results.

- **Activity 3.2: Integrate Model Prediction**

Convert user inputs into the required format.

Pass inputs to the trained model.

Display the predicted well-being score.

### **Milestone 4: Frontend Development**

- **Activity 4.1: Design the User Interface**

Develop responsive web pages using HTML, CSS, and JavaScript.

Create an input form for development indicators.

Design an attractive results section.

- **Activity 4.2: Display Prediction Results**

Submit user inputs asynchronously.

Display the predicted well-being score.

Show the result in a clean and user-friendly format.

### **Milestone 5: Deployment**

- **Activity 5.1: Local Testing**

Test the application on the local Flask server.

Verify prediction accuracy and frontend functionality.

- **Activity 5.2: Deploy the Application**

Deploy the Flask application on Render.

Test the live application and verify end-to-end functionality.

## Milestone 6: Conclusion

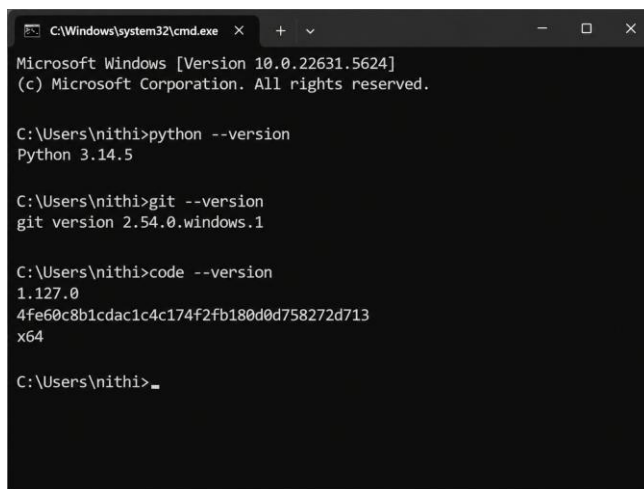
# Milestone 1: Project Setup and Development Environment

### Objective:

The objective of this milestone is to prepare the complete development environment required for building the Comprehensive Measure of Well-Being Prediction System. All necessary software, project folders, libraries, and configurations are set up before starting the application development.

### Activity 1.1: Install Required Software

Python, Visual Studio Code, and Git were installed successfully. These tools provide the required environment for developing, testing, and managing the machine learning and Flask application.



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\nithi>python --version
Python 3.14.5

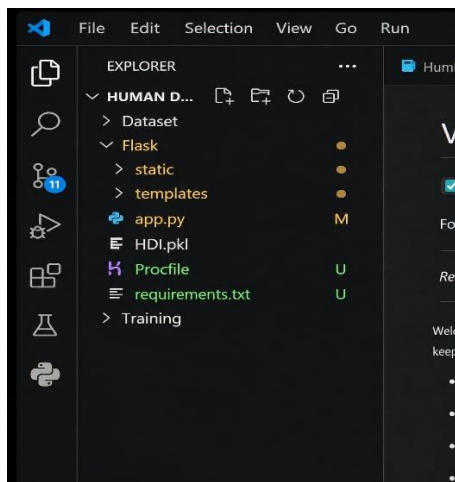
C:\Users\nithi>git --version
git version 2.54.0.windows.1

C:\Users\nithi>code --version
1.127.0
4fe60c8b1cdac1c4c174f2fb180d0d758272d713
x64

C:\Users\nithi>
```

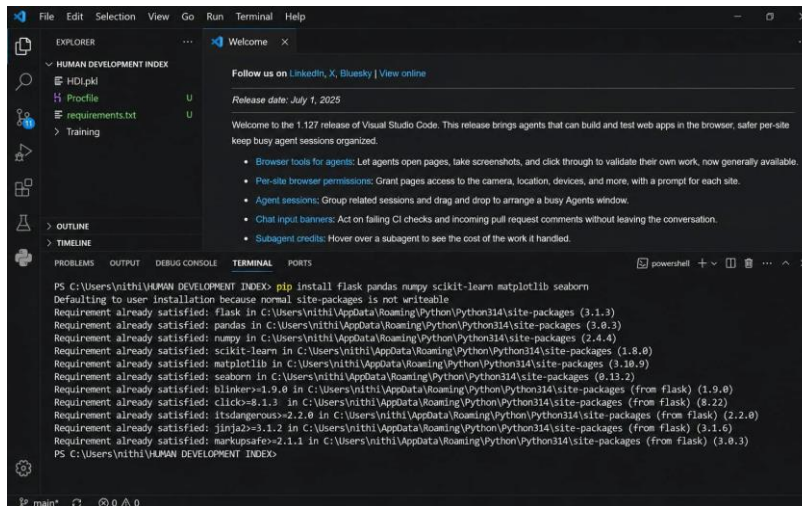
### Activity 1.2: Create Project Folder

A well-organized project folder structure was created, including folders such as templates, static, and files like app.py, HDI.pkl, and requirements.txt to ensure proper project organization.



### Activity 1.3: Install Required Libraries

Essential Python libraries including Flask, Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn were installed using pip. These libraries support data processing, model development, visualization, and web application deployment.

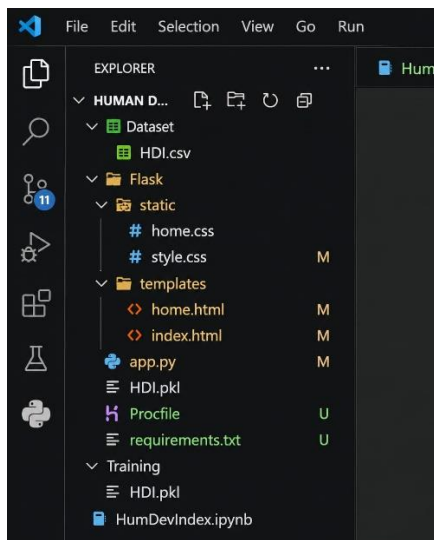


```

PS C:\Users\nithi\HUMAN DEVELOPMENT INDEX> pip install flask pandas numpy scikit-learn matplotlib seaborn
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: flask in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (3.1.1)
Requirement already satisfied: pandas in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (3.0.3)
Requirement already satisfied: numpy in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (2.4.4)
Requirement already satisfied: scikit-learn in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (1.8.0)
Requirement already satisfied: matplotlib in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (3.10.9)
Requirement already satisfied: seaborn in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (0.13.2)
Requirement already satisfied: blinker>=1.9.0 in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (from flask) (8.2.2)
Requirement already satisfied: itsdangerous>=2.2.0 in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (from flask) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (from flask) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in C:\Users\nithi\AppData\Roaming\Python\Python314\site-packages (from flask) (3.0.3)
PS C:\Users\nithi\HUMAN DEVELOPMENT INDEX>
  
```

### Activity 1.4: Setup Development Environment

The complete development environment was configured by organizing all project files, integrating the trained machine learning model, and verifying that all required dependencies were working correctly before implementation.



## Milestone 2: Backend Development

**Objective:**

The objective of this milestone is to develop the backend of the application using Flask and integrate the trained machine learning model for prediction.

### Activity 2.1: Develop Flask Application

A Flask application was developed with routes to display the home page and process prediction requests submitted by the user.

### Activity 2.2: Integrate Machine Learning Model

The trained Well-Being prediction model was loaded using Pickle and integrated into the Flask application to generate predictions based on user inputs.

```

1  from flask import Flask, request, render_template
2  import pickle
3  import numpy as np
4  import os
5
6  print("Current Folder: ", os.getcwd())
7  print("Files:", os.listdir())
8
9  app = Flask(__name__)
10 model = pickle.load(open('HDI.pkl', 'rb'))
11
12 @app.route('/')
13 def home():
14     return render_template('home.html')
15
16 @app.route('/prediction')
17 def prediction():
18     return render_template('index.html')
19
20 @app.route('/predict', methods=['POST'])
21 def predict():
22     features = [float(x) for x in request.form.values()]
23     prediction = model.predict([features])
24     hdi = prediction[0][0]
25
26     if hdi < 0.550:
27         category = "● Low Human Development"
28         recommendations = [
29             "Improve access to quality healthcare.",
30             "Increase school enrollment and literacy.",
31             "Develop basic infrastructure.",
32             "Create more employment opportunities.",
33             "Improve access to clean water and sanitation."
34         ]
35
36     elif hdi < 0.700:
37         category = "● Medium Human Development"
38         recommendations = [
39             "Improve higher education quality.",
40             "Strengthen healthcare services.",
41             "Promote vocational and technical training.",
42             "Increase employment opportunities.",
43             "Support rural development programs."
44         ]
45
46     elif hdi < 0.800:
47         category = "● High Human Development"
48         recommendations = [
49             "Invest in research and innovation.",
50             "Improve digital literacy.",
51             "Promote sustainable development.",
52             "Enhance higher education.",
53             "Reduce income inequality."
54         ]
55     else:
56         category = "● Very High Human Development"
57         recommendations = [
58             "Maintain healthcare excellence.",
59             "Promote green technologies.",
60             "Encourage innovation and R&D.",
61             "Improve quality of life.",
62             "Strengthen environmental sustainability."
63         ]
64
65     return render_template('index.html',
66                           prediction_text=f"● Predicted Human Development",
67                           Index (HDI): (prediction[0][0]:.4f)',
68                           category=category,
69                           recommendations=recommendations)
70
71 if __name__ == "__main__":
72     app.run(debug=True)

```

## Milestone 3: Frontend Development

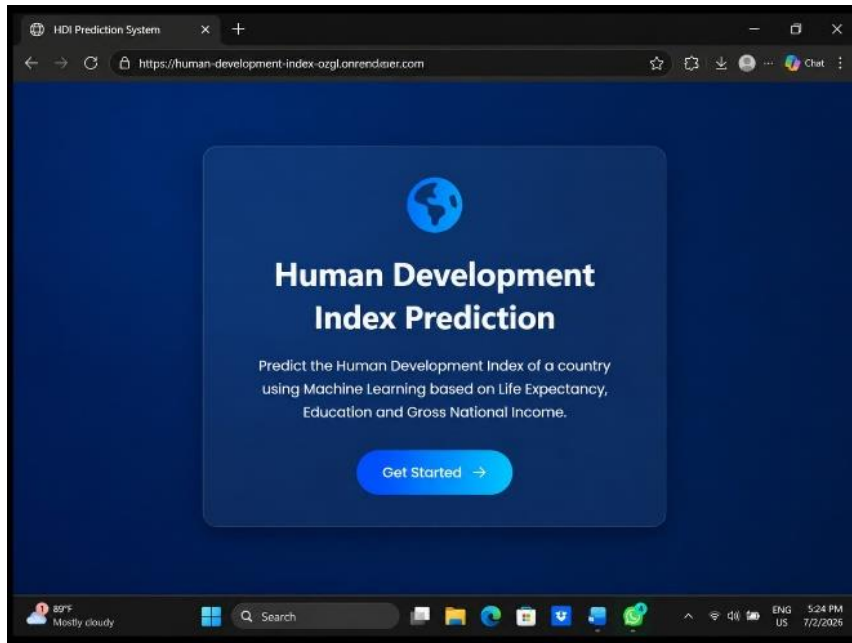
### Objective:

The objective of this milestone is to create a user-friendly interface that allows users to enter input values and view prediction results.

### Activity 3.1: Design User Interface

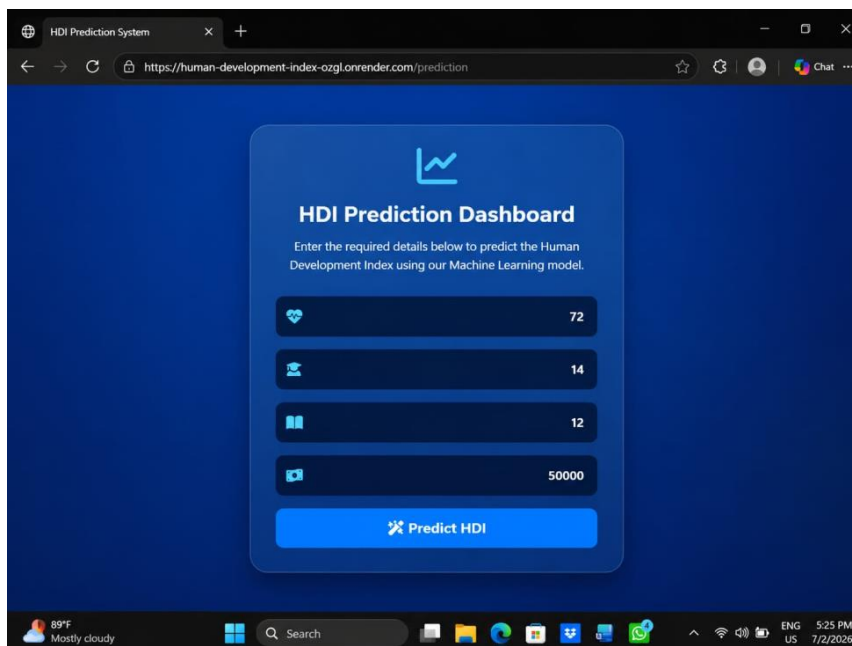
An interactive user interface was designed using HTML and CSS to collect user input required for Well-Being prediction.





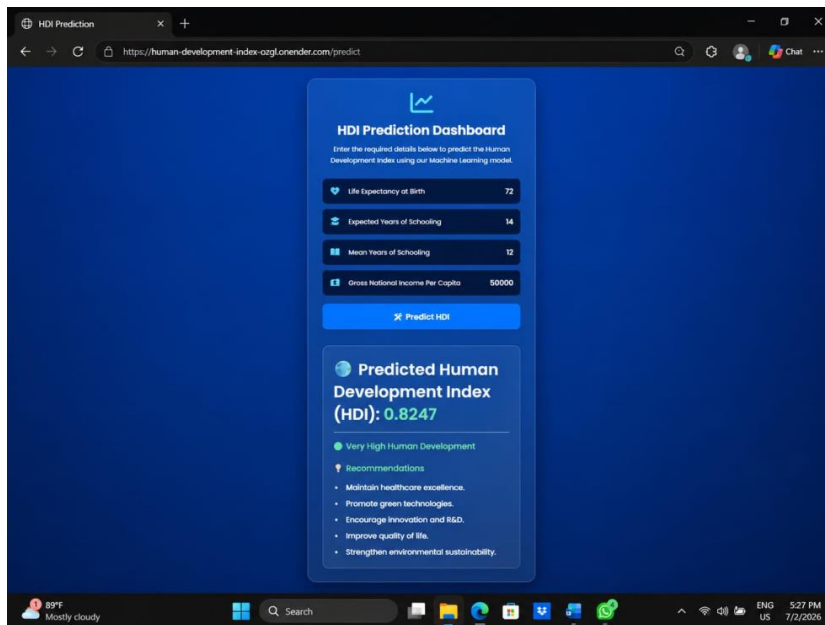
### Activity 3.2: Integrate Frontend with Backend

The frontend was connected with the Flask backend using HTML forms, enabling user input to be processed by the prediction model.



### Activity 3.3: Display Prediction Results

The predicted Comprehensive Well-Being score was displayed clearly on the webpage after successful model prediction.



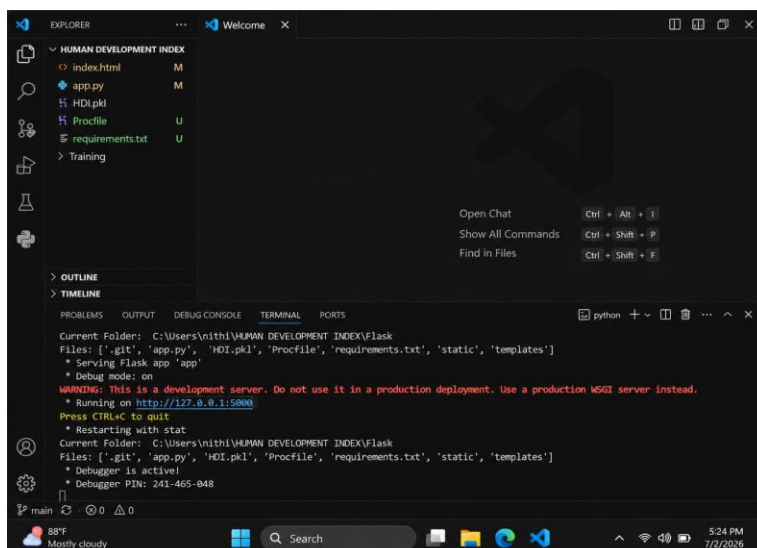
## Milestone 4: Testing

### Objective:

The objective of this milestone is to verify the functionality, accuracy, and reliability of the developed application.

### Activity 4.1: Test Application

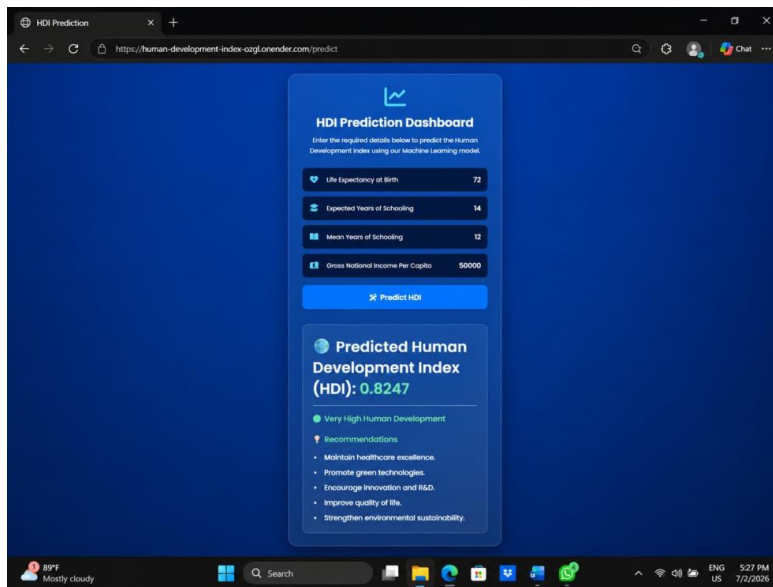
The application was tested locally using the Flask development server to ensure that all features were functioning correctly.





## Activity 4.2: Fix Errors

Any identified issues or errors were analyzed and resolved to improve the overall performance and stability of the application.



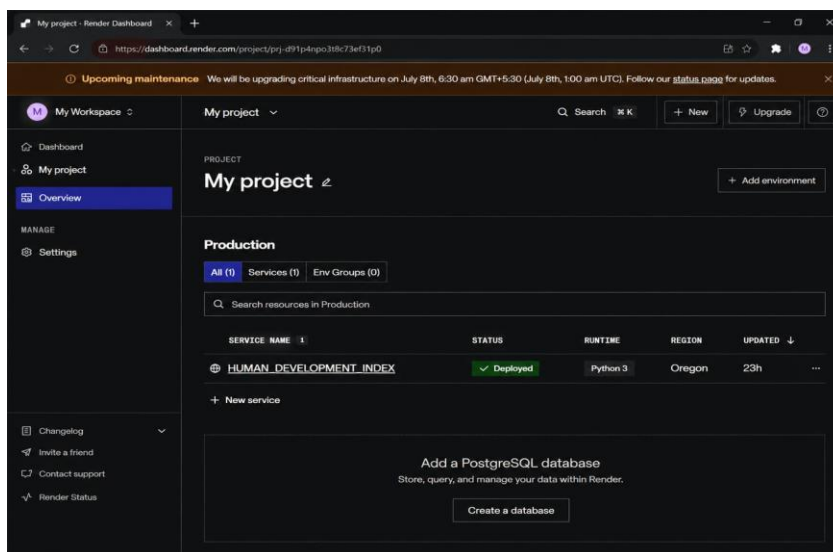
## Milestone 5: Deployment

### Objective:

The objective of this milestone is to deploy the application online and verify its accessibility for end users.

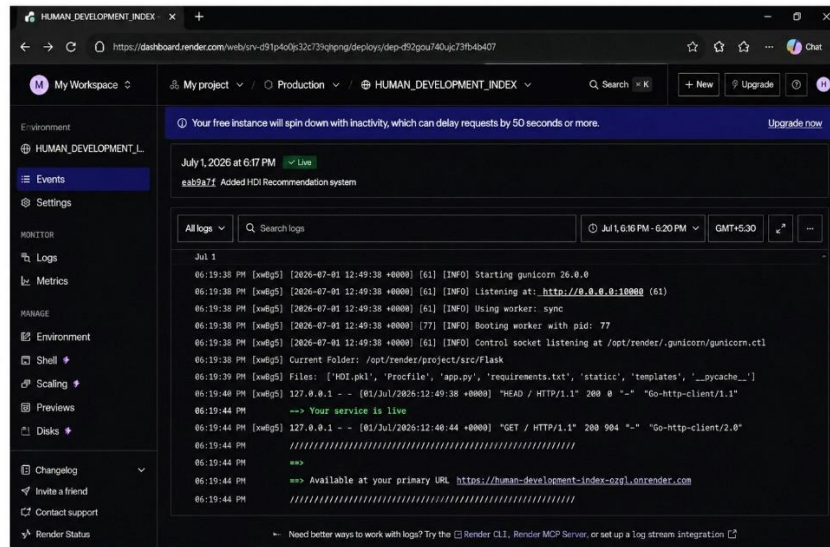
### Activity 5.1: Deploy Application

The Flask application was successfully deployed on Render, making it accessible through a public URL.



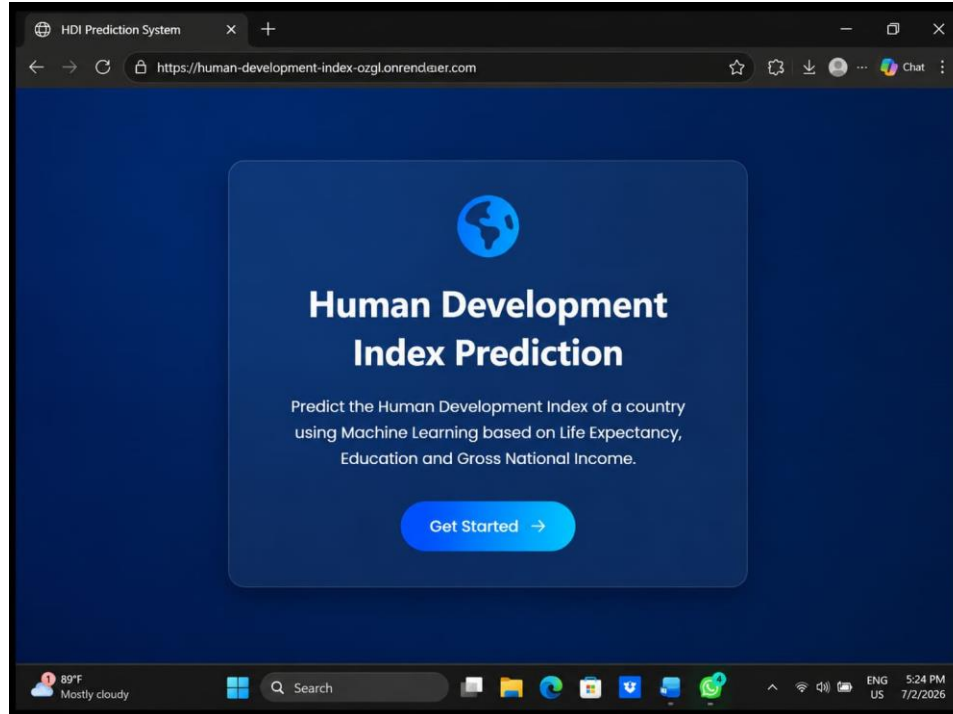
## Activity 5.2: Verify Live Deployment

The deployed application was tested to confirm that all features and prediction functionality were working properly.



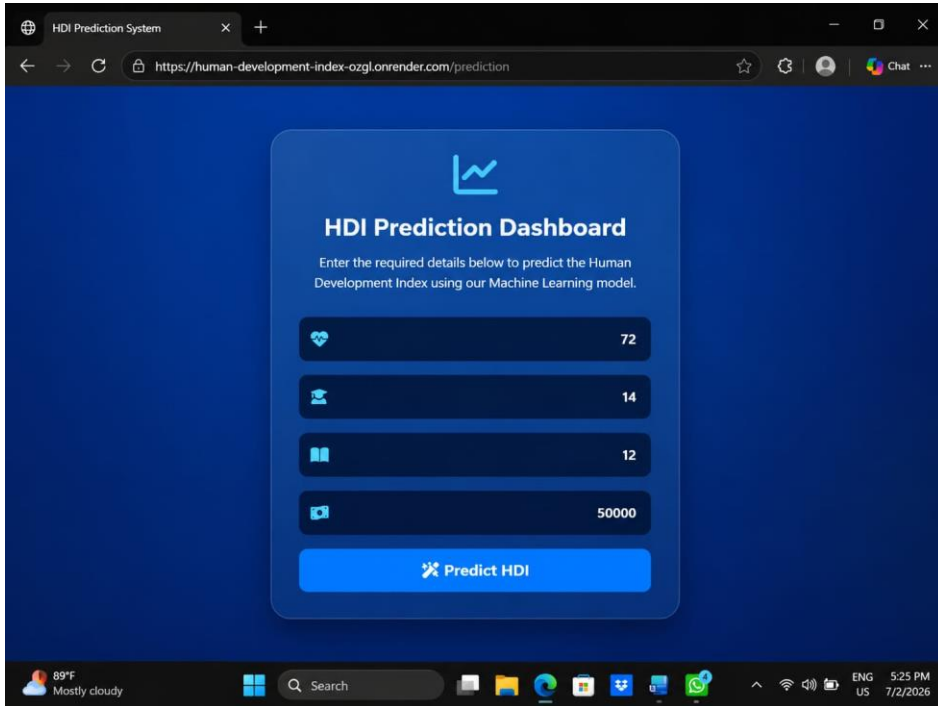
## Exploring the Website's Web Pages:

### Home / Get Started Page:



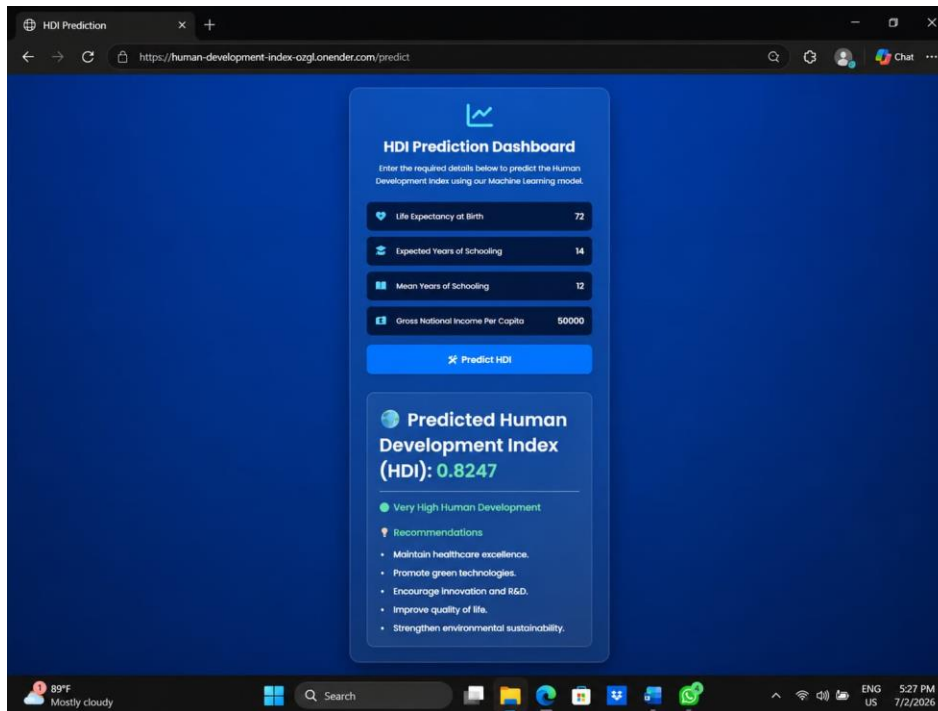
**Description:** The deployed application was accessed using the public Render URL. The Home Page loaded successfully, displaying the Human Development Index Prediction interface with the Get Started option. This confirmed that the application was live, accessible over the internet, and functioning correctly after deployment.

## Input Section:



**Description:** The deployed application was successfully accessed through the public Render URL, and the HDI Prediction Dashboard loaded correctly. The input form accepted the required values for Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income per Capita, confirming that the user interface and input functionality were working properly.

## Results Section:



**Description:** The application successfully processed the user-provided input values and generated the predicted Human Development Index (HDI) score. Based on the predicted value, the system displayed the corresponding Human Development category along with personalized recommendations, confirming that the deployed application was functioning accurately and producing reliable prediction results.

## Conclusion:

The A Comprehensive Measure of Well-Being Prediction Using Machine Learning project demonstrates how Artificial Intelligence and Machine Learning can be effectively used to evaluate and predict a country's overall well-being based on key socio-economic, health, education, and economic indicators. The application integrates a trained Machine Learning model with a Flask-based web interface, enabling users to input relevant development parameters and receive accurate well-being predictions in real time.

Throughout the project, data preprocessing, feature selection, model training, evaluation, web application development, and deployment were successfully completed using Python, Scikit-learn, Flask, HTML, CSS, and Render. The system provides an easy-to-use platform for researchers, students, policymakers, and analysts to estimate well-being levels and understand the impact of different development indicators.

Looking ahead, the application can be further enhanced by integrating real-time global datasets, interactive visualizations, historical trend analysis, country comparison features, user authentication, and advanced Machine Learning or Deep Learning models to improve prediction accuracy. These future enhancements will make the application a more comprehensive decision-support system for analyzing and promoting sustainable human development and overall well-being.